

Integer I

CS3014 - Estructura de Datos Avanzados

Luciano A. Romero Calla

lromeroc@utec.edu.pe

2026-1

Overview

1. Goals
2. Models
3. Predecesor problem
4. van Emde Boas
5. Tree view
6. Saving space

1.

Goals

Goals

- ▶ Explore integer data structures to solve the predecessor problem.

Goals

- ▶ Explore integer data structures to solve the predecessor problem.
- ▶ Describe two main data structures **van Emde Boas** and **Y-fast** trees.

2.

Models

Models

word

w -bit integer $\in U = \{0, 1, 2, 3, \dots, u = 2^w - 1\}$

Models

word

w -bit integer $\in U = \{0, 1, 2, 3, \dots, u = 2^w - 1\}$

transdichotomous RAM

bridging two worlds machine/problem, $w \geq \lg S$

Models

word

w -bit integer $\in U = \{0, 1, 2, 3, \dots, u = 2^w - 1\}$

transdichotomous RAM

bridging two worlds machine/problem, $w \geq \lg S$

word RAM

transdichotomous RAM with C-style operations

Models

word

w -bit integer $\in U = \{0, 1, 2, 3, \dots, u = 2^w - 1\}$

transdichotomous RAM

bridging two worlds machine/problem, $w \geq \lg S$

word RAM

transdichotomous RAM with C-style operations

cell probe

count # memory reads & writes, computation-free,
useful for lower bounds

Models

word

w -bit integer $\in U = \{0, 1, 2, 3, \dots, u = 2^w - 1\}$

transdichotomous RAM

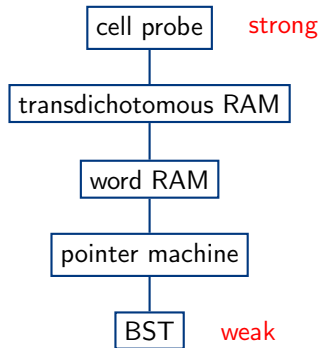
bridging two worlds machine/problem, $w \geq \lg S$

word RAM

transdichotomous RAM with C-style operations

cell probe

count # memory reads & writes, computation-free,
useful for lower bounds



3.

Predecesor problem

Predecesor problem

problem

maintain a set S of n words

- ▶ $\text{insert}(x \in U)$
- ▶ $\text{delete}(x \in S)$
- ▶ $\text{predecessor}(x \in U): \max\{y \in S | y < x\}$
- ▶ $\text{successor}(x \in U): \max\{y \in S | y > x\}$

Predecesor problem

problem

maintain a set S of n words

- ▶ $\text{insert}(x \in U)$
- ▶ $\text{delete}(x \in S)$
- ▶ $\text{predecessor}(x \in U): \max\{y \in S | y < x\}$
- ▶ $\text{successor}(x \in U): \max\{y \in S | y > x\}$

BST

comparison model: $O(\lg n)$ /op optimal

Predecessor problem

problem

maintain a set S of n words

- ▶ $\text{insert}(x \in U)$
- ▶ $\text{delete}(x \in S)$
- ▶ $\text{predecessor}(x \in U): \max\{y \in S \mid y < x\}$
- ▶ $\text{successor}(x \in U): \max\{y \in S \mid y > x\}$

BST

comparison model: $O(\lg n)$ /op optimal

word RAM

- ▶ van Emde Boas: $O(\lg w)$ /op, $\Theta(u)$ space
- ▶ Y-fast trees: $O(\lg w)$ /op, $\Theta(n)$ space
- ▶ fusion trees: $O(\log_w n)$ /op, $\Theta(n)$ space

Predecesor problem

problem

maintain a set S of n words

- ▶ $\text{insert}(x \in U)$
- ▶ $\text{delete}(x \in S)$
- ▶ $\text{predecessor}(x \in U): \max\{y \in S \mid y < x\}$
- ▶ $\text{successor}(x \in U): \max\{y \in S \mid y > x\}$

BST

comparison model: $O(\lg n)/\text{op}$ optimal

word RAM

- ▶ van Emde Boas: $O(\lg w)/\text{op}$, $\Theta(u)$ space
- ▶ Y-fast trees: $O(\lg w)/\text{op}$, $\Theta(n)$ space
- ▶ fusion trees: $O(\log_w n)/\text{op}$, $\Theta(n)$ space

cell probe lower bound

$O(n \text{ poly} \lg n)$ space, $\Omega \left(\min \left\{ \log_w n, \frac{\lg w}{\lg \frac{\lg w}{\lg \lg n}} \right\} \right)$

Predecessor problem

problem

maintain a set S of n words

- ▶ $\text{insert}(x \in U)$
- ▶ $\text{delete}(x \in S)$
- ▶ $\text{predecessor}(x \in U): \max\{y \in S \mid y < x\}$
- ▶ $\text{successor}(x \in U): \max\{y \in S \mid y > x\}$

BST

comparison model: $O(\lg n)$ /op optimal

word RAM

- ▶ van Emde Boas: $O(\lg w)$ /op, $\Theta(u)$ space
- ▶ Y-fast trees: $O(\lg w)$ /op, $\Theta(n)$ space
- ▶ fusion trees: $O(\log_w n)$ /op, $\Theta(n)$ space

cell probe lower bound

$O(n \text{ poly} \lg n)$ space, $\Omega \left(\min \left\{ \log_w n, \frac{\lg w}{\lg \frac{\lg w}{\lg \lg n}} \right\} \right)$

pointer machine

- ▶ word specified by node pointer
- ▶ van Emde Boas: $O(\lg \lg u)$ /op, $\Theta(u)$ space
- ▶ lower bound: $\Omega(\lg \lg u)$ /op, $\Omega(u)$ space

4.

van Emde Boas

4.1 Definition

4.2 Operations

Definition

idea

$$T(u) = T(\sqrt{u}) + O(1)$$

Definition

idea

$$T(u) = T(\sqrt{u}) + O(1)$$

hierarchical coordinates

word $x = \langle c, i \rangle$

Definition

idea

$$T(u) = T(\sqrt{u}) + O(1)$$

hierarchical coordinates

word $x = \langle c, i \rangle$

recursive vEB V of size u

► $V.\text{cluster}[i] = \text{vEB of size } \sqrt{u}$

Definition

idea

$$T(u) = T(\sqrt{u}) + O(1)$$

hierarchical coordinates

word $x = \langle c, i \rangle$

recursive vEB V of size u

- ▶ $V.\text{cluster}[i] = \text{vEB of size } \sqrt{u}$
- ▶ $V.\text{summary}[i] = \text{vEB of size } \sqrt{u}$

Definition

idea

$$T(u) = T(\sqrt{u}) + O(1)$$

hierarchical coordinates

word $x = \langle c, i \rangle$

recursive vEB V of size u

- ▶ $V.\text{cluster}[i] = \text{vEB of size } \sqrt{u}$
- ▶ $V.\text{summary}[i] = \text{vEB of size } \sqrt{u}$
- ▶ $V.\text{min} = \text{minimum element in } V$

Definition

idea

$$T(u) = T(\sqrt{u}) + O(1)$$

hierarchical coordinates

word $x = \langle c, i \rangle$

recursive vEB V of size u

- ▶ $V.\text{cluster}[i] = \text{vEB of size } \sqrt{u}$
- ▶ $V.\text{summary}[i] = \text{vEB of size } \sqrt{u}$
- ▶ $V.\text{min} = \text{minimum element in } V$
- ▶ $V.\text{max} = \text{maximum element in } V$

Definition

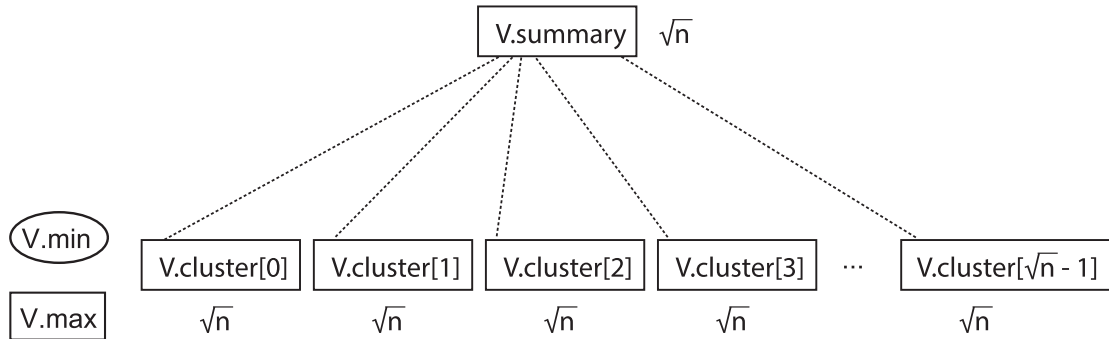


Figure 1: van Emde Boas structure¹

¹Demaine 2021.

Operations

Successor($V, x = \langle c, i \rangle$)

- ▶ if $x < V.min$ return $V.min$
- ▶ if $i < V.cluster[c].max$
return $\langle c, \text{Successor}(V.cluster[c], i) \rangle$
- ▶ else $c' = \text{Successor}(V.summary, c)$
return $\langle c', V.cluster[c'].min \rangle$

5.

Tree view

5.1 Indirection

Tree view

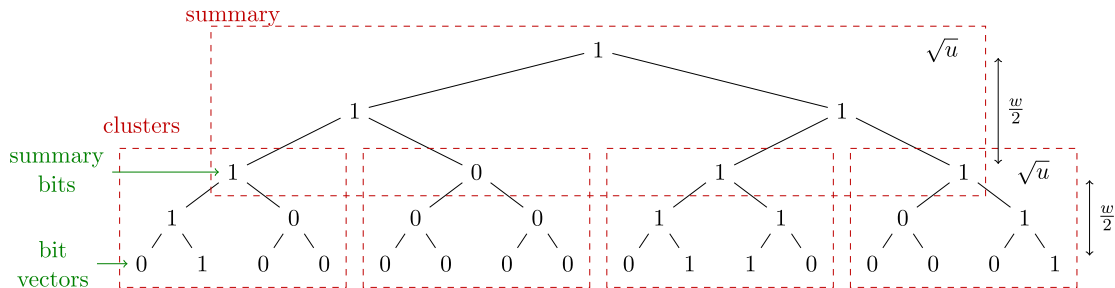


Figure 2: van Emde Boas tree view²

²Demaine 2021.

Indirection

- ▶ from $O(\lg w)$ query, $O(w)$ update DS

Indirection

- ▶ from $O(\lg w)$ query, $O(w)$ update DS
- ▶ reduce to $O(\lg w)$ update: split n elements into chunks of size $\Theta(w)$

6.

Saving space

6.1 X-fast trees

6.2 Y-fast trees

Saving space

- ▶ don't store empty clusters vEB

Saving space

- ▶ don't store empty clusters vEB
- ▶ $V.clusters = \text{hash table}$

Saving space

- ▶ don't store empty clusters vEB
- ▶ $V.clusters = \text{hash table}$
- ▶ charge each table entry to min in child

Saving space

- ▶ don't store empty clusters vEB
- ▶ $V.clusters = \text{hash table}$
- ▶ charge each table entry to min in child
- ▶ space bound $\Theta(n)$ via indirection

X-fast trees

- ▶ don't store 0s

X-fast trees

- ▶ don't store 0s
- ▶ store a hash table of root-to-1 path

X-fast trees

- ▶ don't store 0s
- ▶ store a hash table of root-to-1 path
- ▶ $O(\lg w)$ query binary search

X-fast trees

- ▶ don't store 0s
- ▶ store a hash table of root-to-1 path
- ▶ $O(\lg w)$ query binary search
- ▶ $\Theta(w)$ update

X-fast trees

- ▶ don't store 0s
- ▶ store a hash table of root-to-1 path
- ▶ $O(\lg w)$ query binary search
- ▶ $\Theta(w)$ update
- ▶ $\Theta(nw)$ space

Y-fast trees

► = X-fast trees

Y-fast trees

▶ = X-fast trees

▶ + indirection

Y-fast trees

- ▶ = X-fast trees
- ▶ + indirection
- ▶ $O(\lg w)$ query binary search

Y-fast trees

- ▶ = X-fast trees
- ▶ + indirection
- ▶ $O(\lg w)$ query binary search
- ▶ $O(\lg w)$ update amortized

Y-fast trees

- ▶ = X-fast trees
- ▶ + indirection
- ▶ $O(\lg w)$ query binary search
- ▶ $O(\lg w)$ update amortized
- ▶ $O(\lg w)$ space

Thanks



References



Demaine, Erik (2021). *6.851: Advanced Data Structures (Spring'21)*. URL: <https://courses.csail.mit.edu/6.851/spring21/>.

Acknowledgements

The course slides are based on the lectures from previous editions and on similar lectures elsewhere.
List of credits: Erick Demaine, Keith Schwarz